

Foundations Continued: RL Post Training

CSE 5539: Special Topics in Large Language Models

<https://go.osu.edu/cse-5539-fall-2026>



THE OHIO STATE UNIVERSITY

Logistics

- Project proposal due Sept 25 at 11.59pm.
 - Come to my office hours today or next Monday if you want feedback on your idea before submission.

Recap

- Basics of pretraining / scaling
 - In-context learning / chain of thought prompting
- Post training
 - SFT / instruction tuning
- Paper presentation
 - General idea: synthetic data for reasoning SFT from a pretrained model

Today: Reinforcement learning based post-training

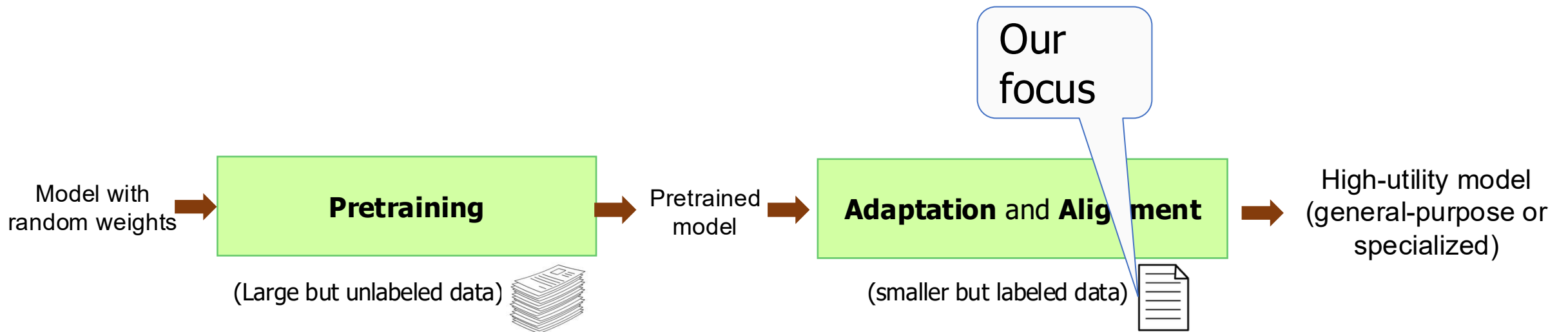
- Basics of reinforcement learning
 - Policy gradients
 - REINFORCE and its challenges
- Training LMs with RL with verifiable rewards
- Training LMs with RL with human feedback

Post Training

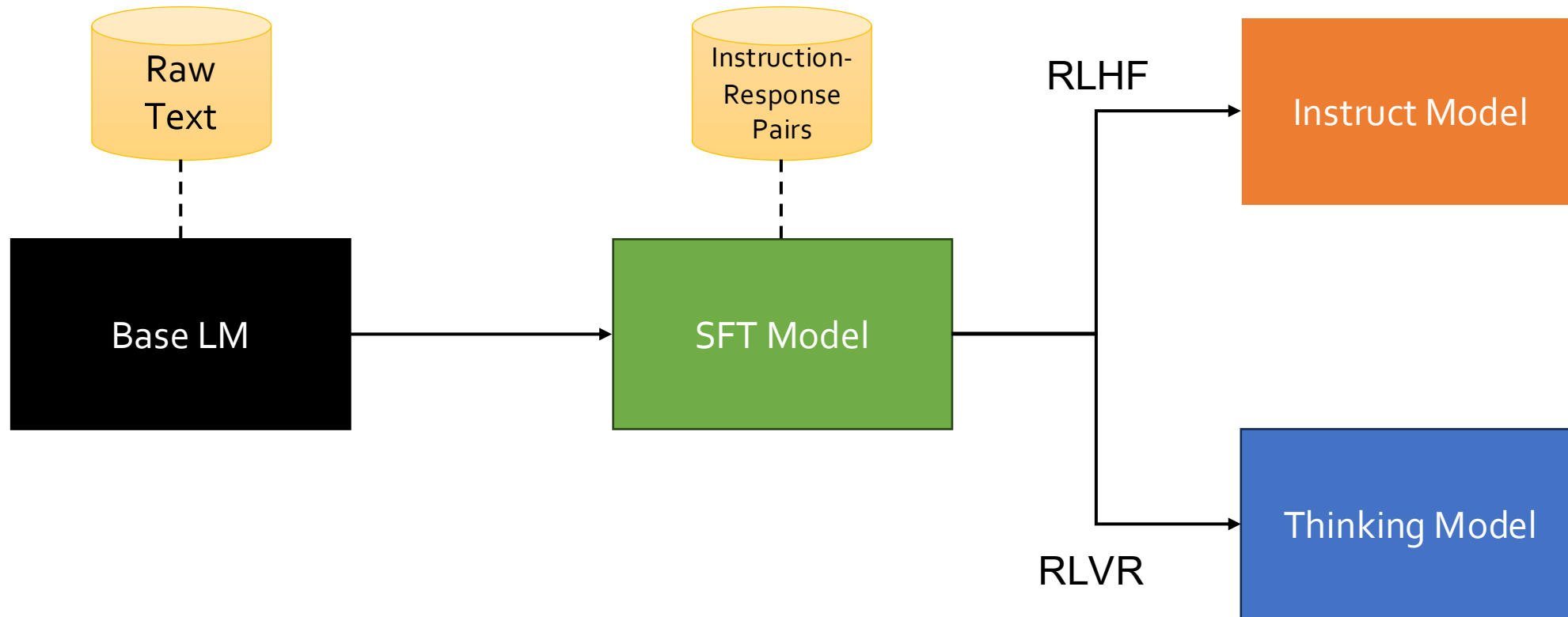
or Alignment

[Mis]Alignment in Language Models

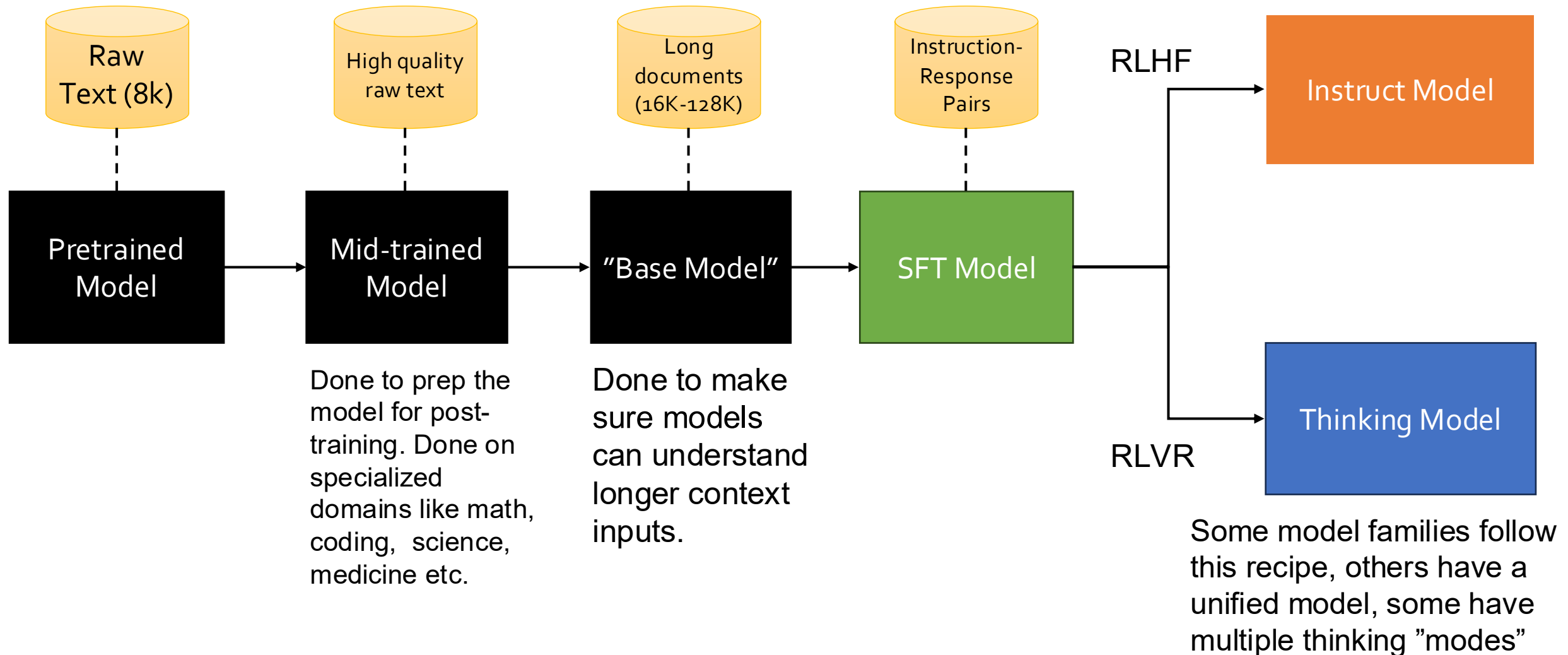
- There is a mismatch between what **pre-trained** models can do and what we want.
- Addressing this gap is the focus of “alignment” research.



Overview of post-training

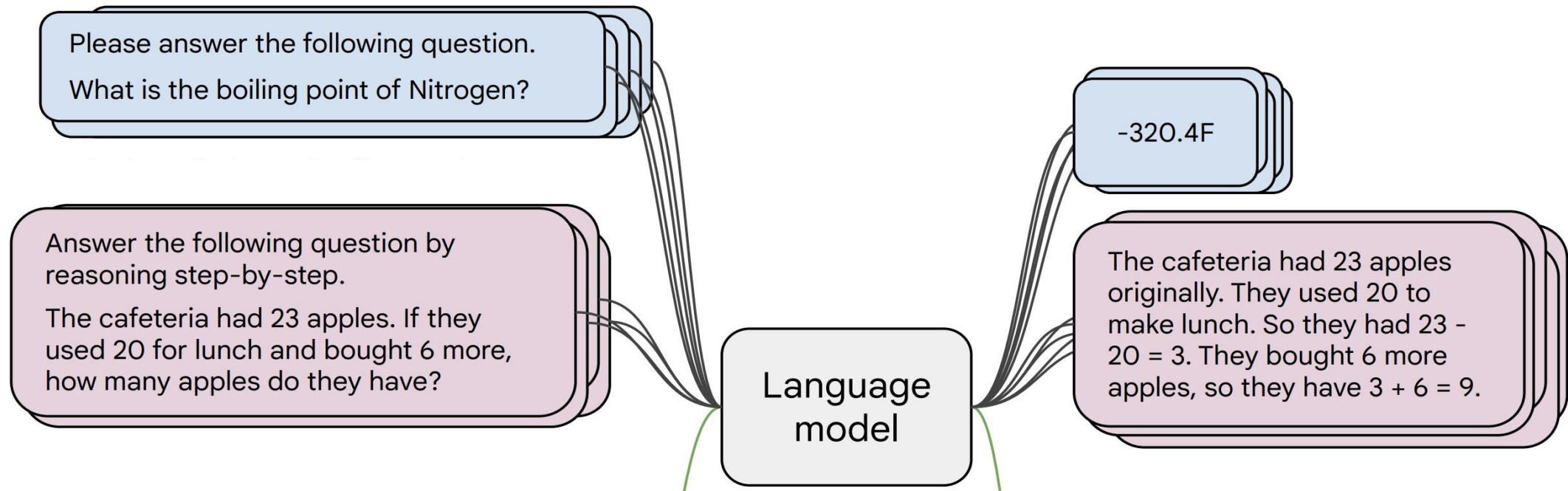


A more accurate modern LM training pipeline



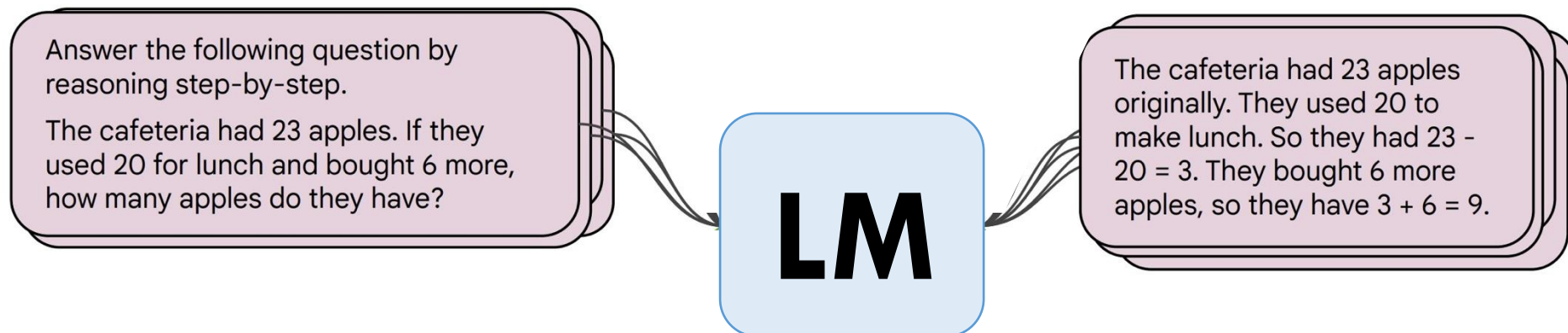
Instruction-tuning / SFT

- **Finetuning** pre-trained LMs to map **instructions** to their corresponding **responses**.



Limits of SFT

1. Difficult to collect diverse data – could be mitigated by bootstrapping from models themselves (like STaR)
2. Resulting models may not be good at open-ended generation tasks.
 - Incentivizes word-by-word rote learning => The resulting LM's **generality/creativity** is bounded by that of **their supervision data**.



Limits of SFT

1. Difficult to collect diverse data.
2. Resulting models may not be good at open-ended generation tasks.
 - Incentivizes word-by-word rote learning => The resulting LM's **generality/creativity** is bounded by that of **their supervision data**.
3. Resulting models may hallucinate more regularly.
 - Labeled data is collected agnostic to the LM's knowledge => there might be a mismatch between labeled data and LM knowledge.

Aligning Language Models: Reinforcement Learning

Reinforcement Learning: Intuition

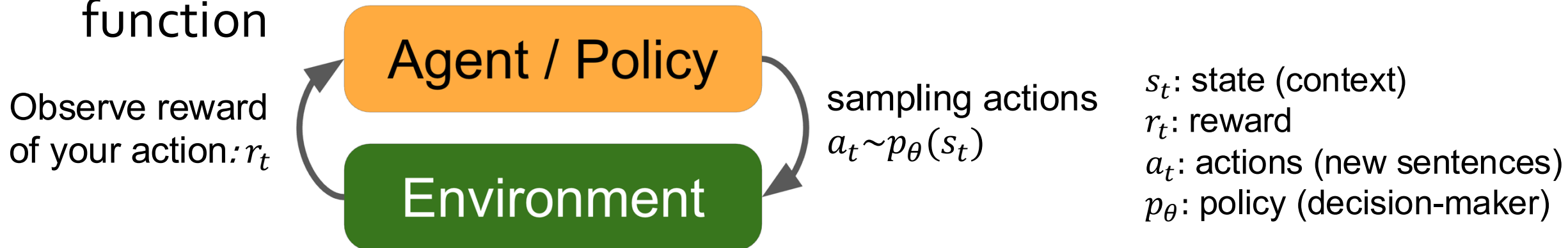
Action here: generating responses/token



Reward: a scalar value serving as a proxy for if the agent does what it's supposed to.

Reinforcement Learning: Formalism

- An agent **interacts** with an environment by taking **actions**
- The environment returns a **reward** for the **action** and a **new state** (representation of the world at that moment).
- Agent uses a **policy function** to choose an action at a given **state**.
- We need to figure out: (1) reward function and (2) the policy function



Reinforcement Learning in LMs

Two major paradigms based on which rewards are used.

1. Reinforcement Learning from Human Feedback (human preferences are converted to rewards – inherently subjective) – RLHF
2. Reinforcement Learning from Verifiable Rewards (such as answer correctness, similar to the STaR paper) -- RLVR

RLHF became the major paradigm starting 2021, starting 2024/2025, most major labs have a thinking model which is trained with RLVR. Most production models is likely a mix of both (and more).

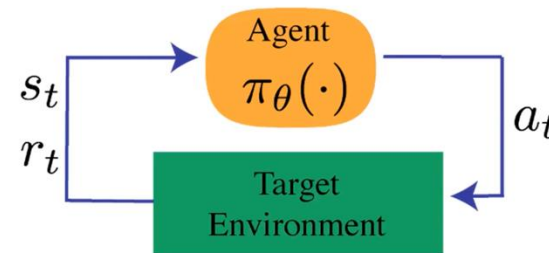
RLVR

RLVR Setup



- Policy Model: A language model, can either just be pretrained or SFTed (typically)
 - Given a prompt, this model is expected to first generate a reasoning trace followed by the final response.
 - $X \rightarrow \langle \text{think} \rangle \text{reasoning trace} \dots \langle / \text{think} \rangle \langle \text{response} \rangle \text{actual response} \langle / \text{response} \rangle$
- The model receives a reward based on the correctness on the content in the $\langle \text{response} \rangle \langle / \text{response} \rangle$ tags.
- The RL process updates the model to maximize the expected reward.

Notation



- a_t : action (next token(s) generated)
- s_t : state (text in the model's context / prompt)
- $p_{\theta}(a|s)$: policy function, parameterized by θ ; distribution over actions at state s (our language model which we want to optimize)
- r_t : reward associated with a given action and state.

There are other notations we will gradually introduce like value model, baselines etc, and some we will skip for simplicity such as Return, Q-function. Please check [RLHF textbook](#) for details.

Reinforcement Learning from Verifiable Rewards

- Imagine a reward function: $R(a; \text{prompt}) \in \mathbb{R}$ for any output a to a prompt.

- Good generation is equivalent to finding reward-maximizing outputs:

$$\mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- What we need to do:

- Find the best generative model p_{θ} that maximizes the expected reward:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$



Step 1: Reward function R

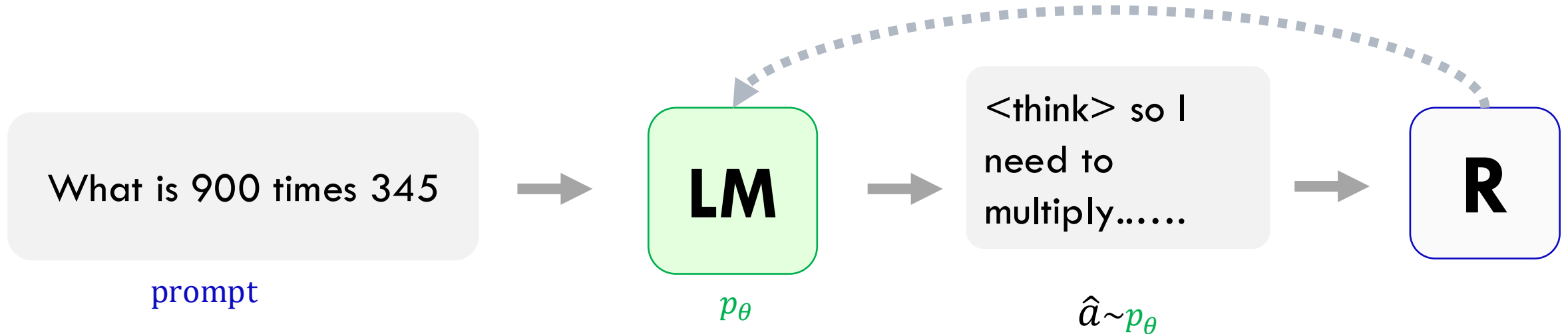
- In RLVR, the most basic reward is typically a binary value (other variants are also used):
 - 1 if the model answer is correct.
 - 0 if the model answer is wrong.
- How to measure correctness?
 - Today we will only talk about tasks whose correctness is verifiable – such as math reasoning (by simply matching the output with the correct answer) or code generation (where compilation errors / test cases can verify correctness of code).
 - Future: approximate correctness with rubric-based rewards / other means.

Step 2: Optimizing the Policy Function



- Policy function := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this?

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$



Step 2: Optimizing the Policy Function



- Policy function := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this?

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

How do we estimate the gradient of this expectation?

Notice that R is not directly dependent on θ . (You can't compute its grad with respect to θ)

- Turns out that we can write this “gradient of expectation” to a simpler form.

Policy Gradient [Williams, 1992]



- How do we change our LM parameters θ to maximize this?

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- With a bit of math, this can be approximated as Monte Carlo samples from $p_{\theta}(a)$:

$$\nabla_{\theta} \mathbb{E}_{a \sim p_{\theta}} [R(a; \text{prompt})] \approx \frac{1}{n} \sum_{i=1}^n R(a_i; \text{prompt}) \nabla_{\theta} \log p_{\theta}(a_i; \text{prompt})$$

- This is “**policy gradient**”, an approach for estimating and optimizing this objective.
- Oversimplified. For full treatment, see [RL textbook](#).

Policy Gradient [Williams, 1992]



Note, $R(s; \text{prompt})$ could be any arbitrary, non-differentiable reward function that we design.

- This gives us the following update rule:

$$\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{|\text{samples}| \times |\text{prompts}|} \sum_{p \in \text{prompts}} \sum_{a_i \sim p_\theta(p)} R(a_i; p) \nabla_\theta \log p_\theta(a_i; p)$$

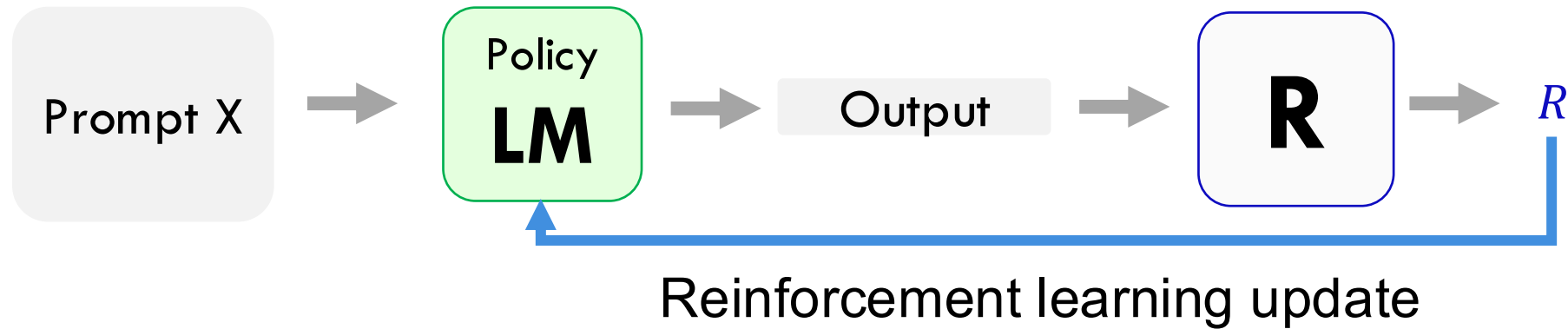
- If $R(a; p)$ is **large**, we take proportionately **large** steps to maximize $p_\theta(a)$
- If $R(a; p)$ is **small**, we take proportionately **small** steps to maximize $p_\theta(a)$

This is why it's called "reinforcement learning":
we reinforce good actions, increasing the chance they happen again.

Summary of basic RL with REINFORCE



- We want to learn a policy (a Language Model) that optimizes against the reward function

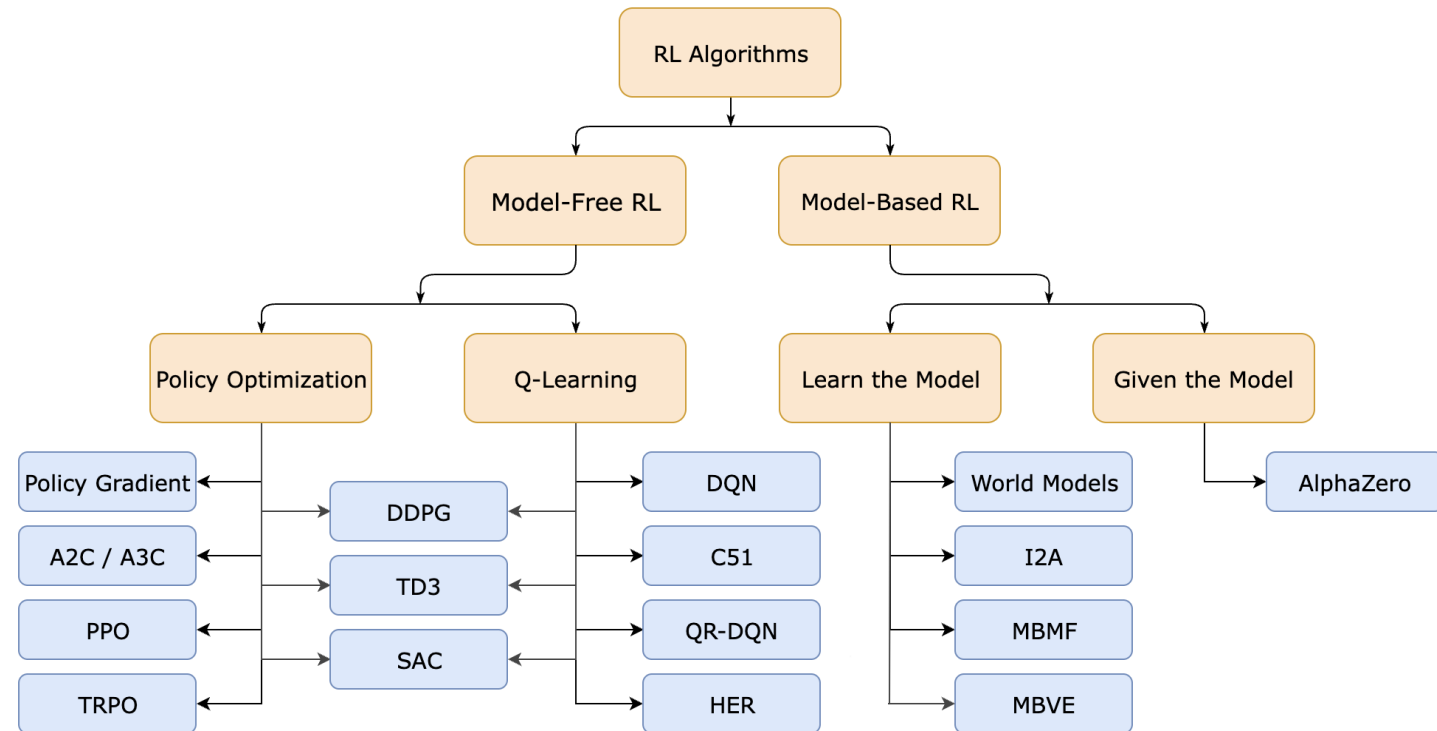


$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^n R(a_i; \mathbf{p}) \nabla_{\theta} \log p_{\theta}(a_i; \mathbf{p})$$

REINFORCE is too simple to work,
what do people *actually* use?

The Bigger Picture

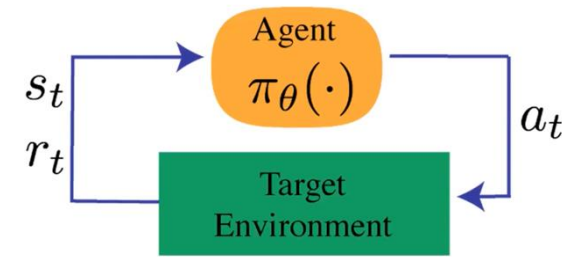
- What we saw was a simple policy gradient algorithm for RL.



Check out Andrew Perrault's courses:
<https://aperrault.github.io/teaching/>

https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

REINFORCE: Challenges



- **Distribution drift:** While the gradient updates maximize the rewards, it may deviate from natural distribution (it may hack its way to high reward).
 - Modification 1: Generate unnatural looking text, this is typically handled by adding a KL divergence loss to the objective, which minimizes too much deviation from the original policy also known as “reference policy”.
- **High variance:** The gradient estimates suffer from high variance.
 - This may lead to destructively large updates and sample inefficiency.

Modification 2: The baseline estimate

- To reduce the variance of the gradient, we subtract a **baseline estimate** $b_t(s_t)$:

$$g^{\text{VR}} = \mathbb{E}[\nabla_{\theta} \log p(a_t|s_t) (r_t - b_t)]$$

- Note, by design, the baseline **depends on states s_t but not the action a_t** .
- Therefore, $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) (r_t - b_t)$ is an “unbiased estimator” of $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) r_t$ (Why?)

What can be a baseline?

$$g^{\text{VR}} = \mathbb{E}_{a \sim p} [\nabla_{\theta} \log p_{\theta}(a_t | s_t) (r_t - b_t)]$$

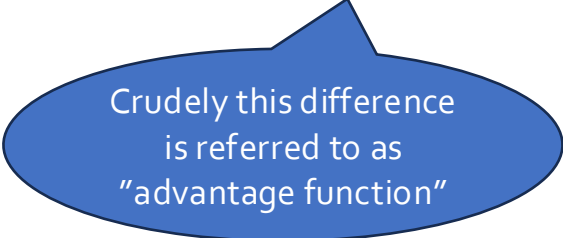
- A “good” baseline is a function that:
 - doesn’t depend on actions.
 - Minimizes variance:
 - $b^*(s_t) = \operatorname{argmin}_b \mathbb{E}_{a \sim \pi} [|g^{\text{VR}} - E[g^{\text{VR}}]|^2]$
 - $b^*(s_t) = V^p(s_t) = \mathbb{E}_{a \sim p} [r_t(s_t, a_t)]$, also known as the **value function**.
- Full derivation in <https://rlhfbook.com/book.pdf#page=57.27>

Value Function as a Baseline

$$g^{\text{VR}} = \mathbb{E}_{a \sim p} [\nabla_{\theta} \log p_{\theta}(a_t | s_t) (r_t - b_t)]$$

- A “good” baseline is a function that:
 - doesn’t depend on actions.
 - Minimizes variance – value function.
- *In practice, value function is learned since it cannot be directly estimated.*
 - $V_{\omega}^p(s)$: value of state s , parameterized by ω ; expected reward from here on under policy p , assuming that we’re at state s .

$$g^{\text{VR}} = \mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r_t - V^{\pi}(s))]$$



Crudely this difference
is referred to as
“advantage function”

Policy Gradient with Advantage Function

- Advantage-based Policy Gradient updates:

$$g^{\text{APG}} = \mathbb{E}[\nabla_{\theta} \log p_{\theta}(a_t | s_t) A_t]$$

- We don't (always) need to compute the absolute benefit of an action, but only how much better it is relative to others (i.e., the **relative advantage** of that action.)
- The advantage function $A^p(s, a)$ of a policy p quantifies **how much better it is to take a specific action a in state s , over a randomly selecting an action according to $p(\cdot | s)$, assuming you act according to p forever after.**
- Now we need an algorithm that updates the policy while estimating the advantages.

Summary so far

- Original: Policy gradient (distribution drift, variances are too high)

$$\max_{\theta} \mathbb{E}[\nabla \log p_{\theta}(a_t | s_t) R_t]$$

- Modification 1: distribution drift – add a KL divergence in the loss to prevent the policy to drift too far away from reference.
- Modification 2: Policy gradient with advantage function to reduce variance.

$$\begin{aligned} \max_{\theta} \mathbb{E}[\nabla \log p_{\theta}(a_t | s_t) A_t] \\ A_t(s, a) = r(s, a) - V^p(s) \end{aligned}$$

Other bag of tricks that constitute “PPO”

- PPO (for a long time) was the standard algorithm for RL in LMs.
 - Prevent distribution drift: using the KL divergence term
 -
 - Use advantage formulation we just discussed
 - Value model is learned along with the policy. We haven’t discussed how, but its pretty straightforward.
 - [New] At each iteration, a bunch of samples are generated, also known as rollouts. This is expensive to generate once and discard. One could do multiple model updates with one batch of rollouts.
 - This makes the updates “off-policy”. The expectation of rewards is not correct anymore.
 - Uses “importance sampling ratio” to get the correct estimate for gradients. Read RL textbook for details.
 - [New] Trust region:
 - Do not move too fast to new weights (stay within the trust region). Make small updates by clipping the importance sampling ratio.
- PPO requires three models in memory: original model (reference), policy model (that you are learning), and a value model (that you also learn).

PPO Objective

$$g = \hat{\mathbb{E}}_t[\nabla \log p_\theta(a_t|s_t) \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & \text{if } r_t(\theta) < 1 - \epsilon \\ 1 + \epsilon & \text{if } r_t(\theta) > 1 + \epsilon \\ r_t(\theta) & \text{else} \end{cases}$$

Recent Extension:

Group Relative Policy Optimization (GRPO)

- PPO has 3 LMs in the mix: value, policy and reference policy and also a fourth one (reward model in case of learned rewards with RLHF, more on this later).
 - Massive memory footprint.
- **GRPO** drops the value model. → Significant reduction of memory usage.
- We had value function in PPO is to estimate "advantage" values.
 - If we find alternative way of estimating advantage, we can drop value function.

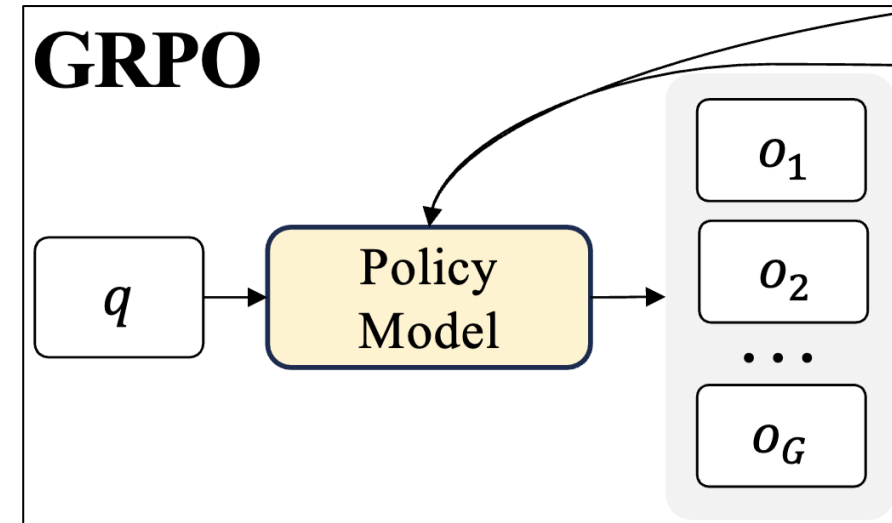
$$\mathbb{E}[\nabla \log p_{\theta}(a_t | s_t) A_t]$$
$$A_t(s, a) = r(s, a) - V^{\pi}(s)$$

GRPO: Key Idea

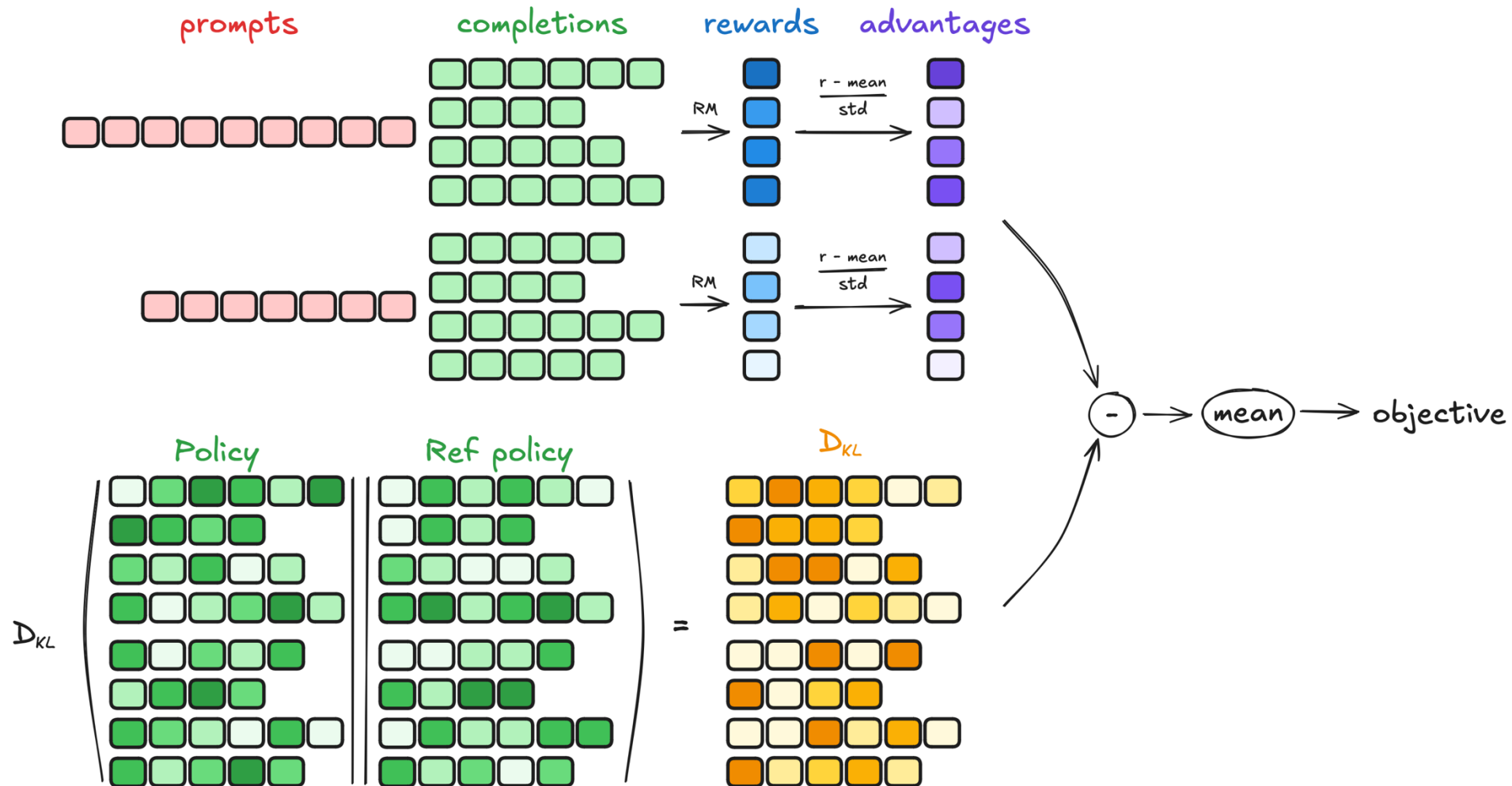
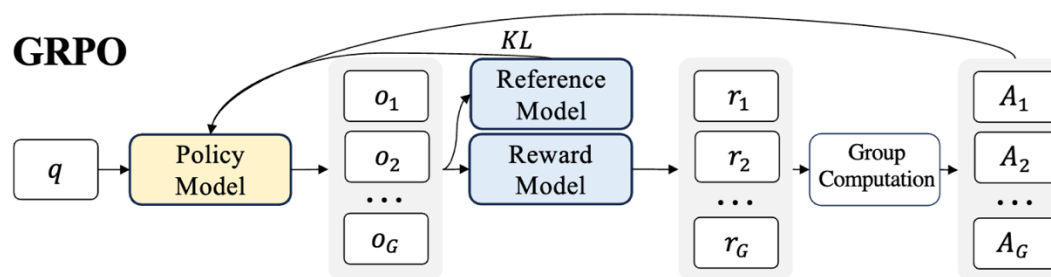
- Execute multiple rollouts from each.
- Given these rollouts, we can estimate the “advantage” function based on **the relative goodness of these responses**.

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

- Advantage of each rollout is simply the gap between its reward compared to the mean reward of other responses, normalized with std.



GRPO



Summary

- RL has many model variants.
- Thus far we have seen:
 - Policy Gradient
 - PPO (barely) – important to remember the idea of value models.
 - GRPO (variants of this will be discussed in future classes as you read papers)
- See implementation of alignment algorithms:
<https://huggingface.co/docs/trl/index>

Reinforcement Learning with *Human Feedback*

Reinforcement Learning from Human Feedback



- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

Expected reward over the course of sampling from our policy (generative model)

$$\mathbb{E}_{\hat{s} \sim p_{\theta}} [R(\hat{s}; \text{prompt})]$$

$p_{\theta}(s)$ is a pre-trained model with params θ we would like to optimize (policy function)

- On the notation:
 - “ $\mathbb{E}$ ” here is an empirical expectation (i.e., average).
 - “ $\sim$ ” indicates sampling from a given distribution.

Reinforcement Learning from Human Feedback

- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

$$\mathbb{E}_{\hat{s} \sim p_{\theta}} [R(\hat{s}; \text{prompt})]$$



- What we need to do:
 - (1) Estimate the reward function $R(s; \text{prompt})$.
 - (2) Find the best generative model p_{θ} that maximizes the expected reward: $\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}} [R(\hat{s}; \text{prompt})]$

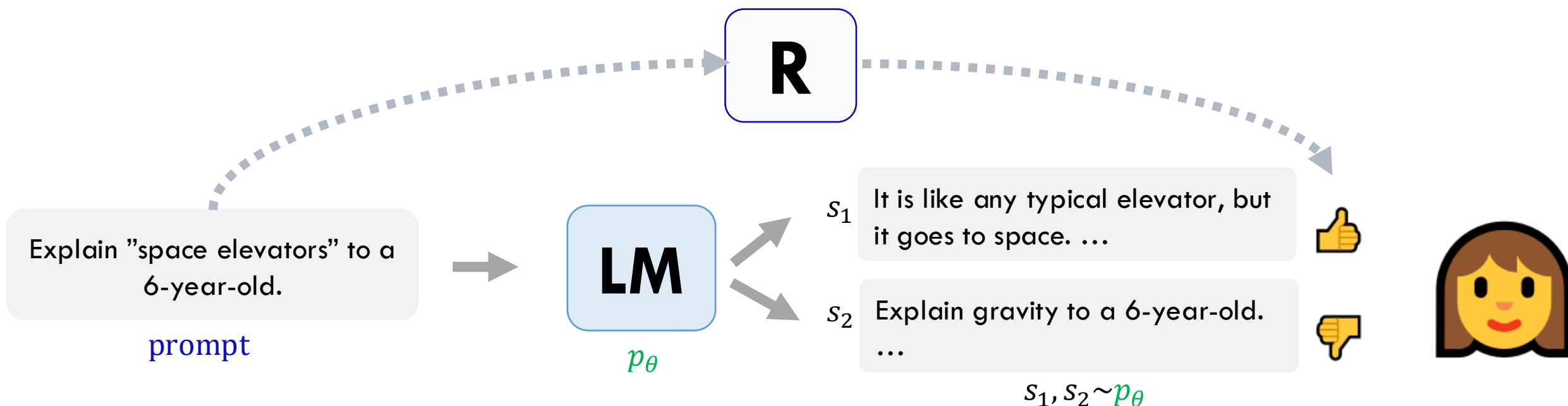
Step 1: Estimating the Reward R

s_1

$$J(\phi) = -\mathbb{E}_{(s^+, s^-)} [\log \sigma(R(s^+; \text{prompt}) - R(s^-; \text{prompt}))]$$

“winning”
sample

“losing”
sample



Step 1: Estimating the Reward R

S_1

$$J(\phi) = -\mathbb{E}_{(s^+, s^-)} [\log \sigma(R(s^+; \text{prompt}) - R(s^-; \text{prompt}))]$$

"winning"
sample

"losing"
sample

R

The reward model returns a scalar reward which should numerically represent the human preference.

Explain "space
elevators" to a 6-year-
old.
prompt

LM

p_θ

s_1

It is like any typical
elevator, but it goes to
space. ...

s_2

Explain gravity to a 6-
year-old. ...

$s_1, s_2 \sim p_\theta$

$R(s_2; \text{prompt}) = 1.2$

$R(s_1; \text{prompt}) = 0.8$

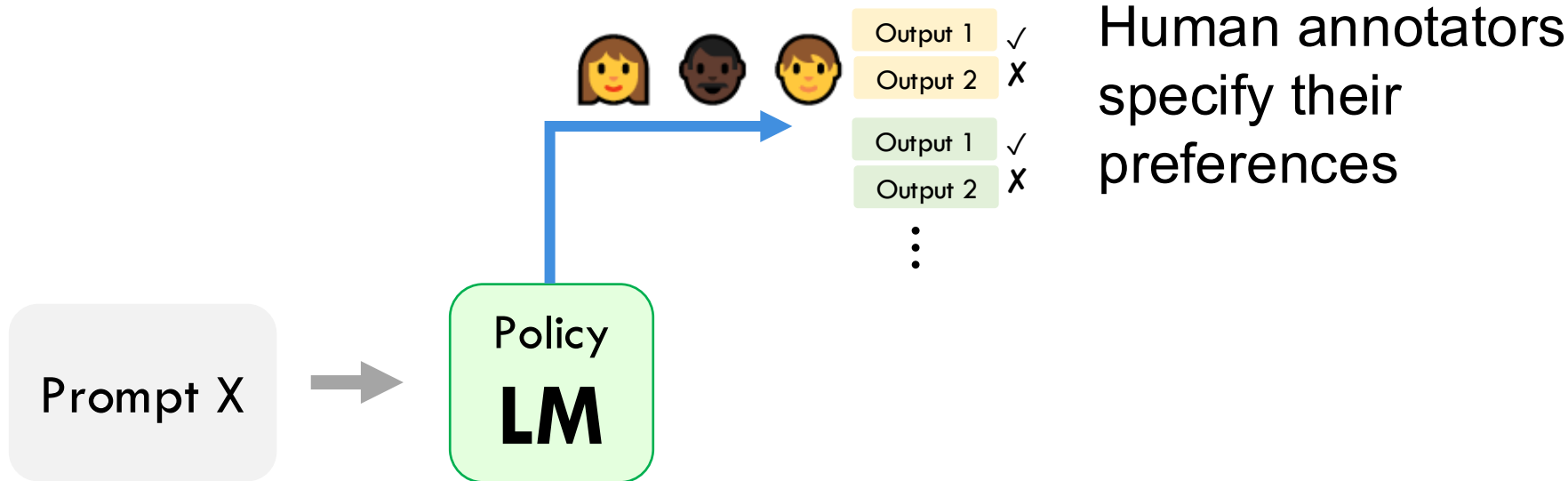
Step 2: Training the Policy

- Using policy gradients methods like (PPO), as discussed above.

Putting it Together



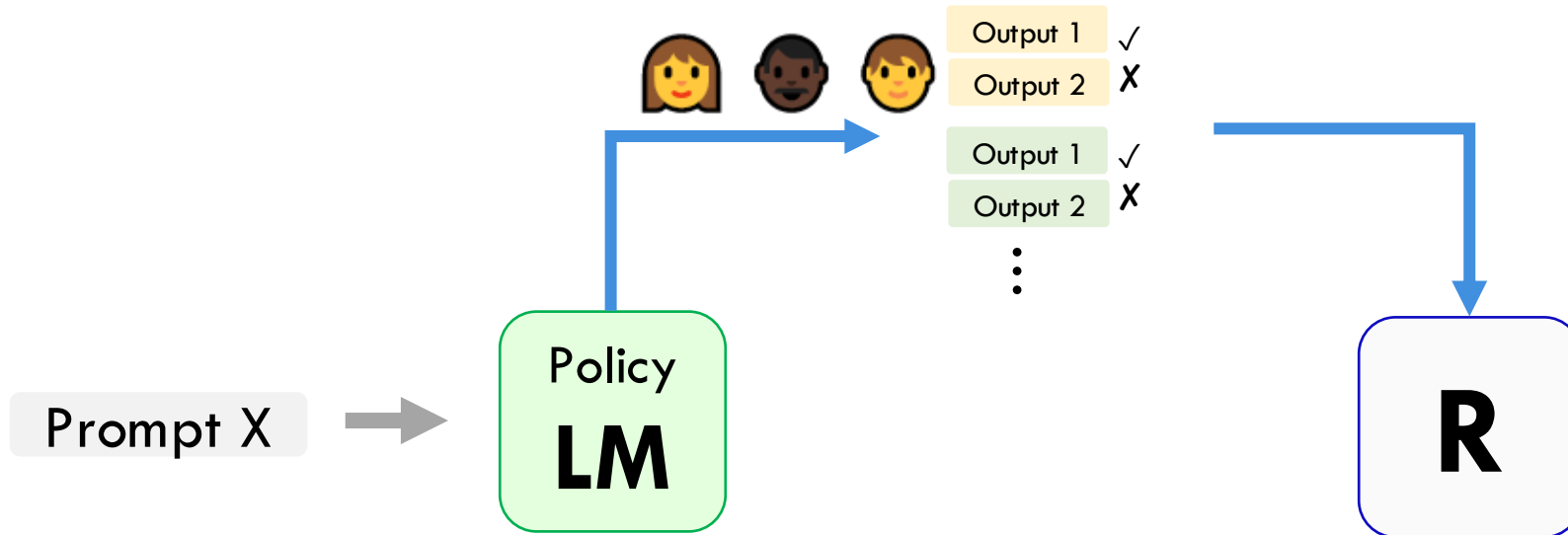
- First collect a dataset of preferences
 - Present multiple outputs to human annotators and ask them to rank the output based on preferability



Putting it Together (2)



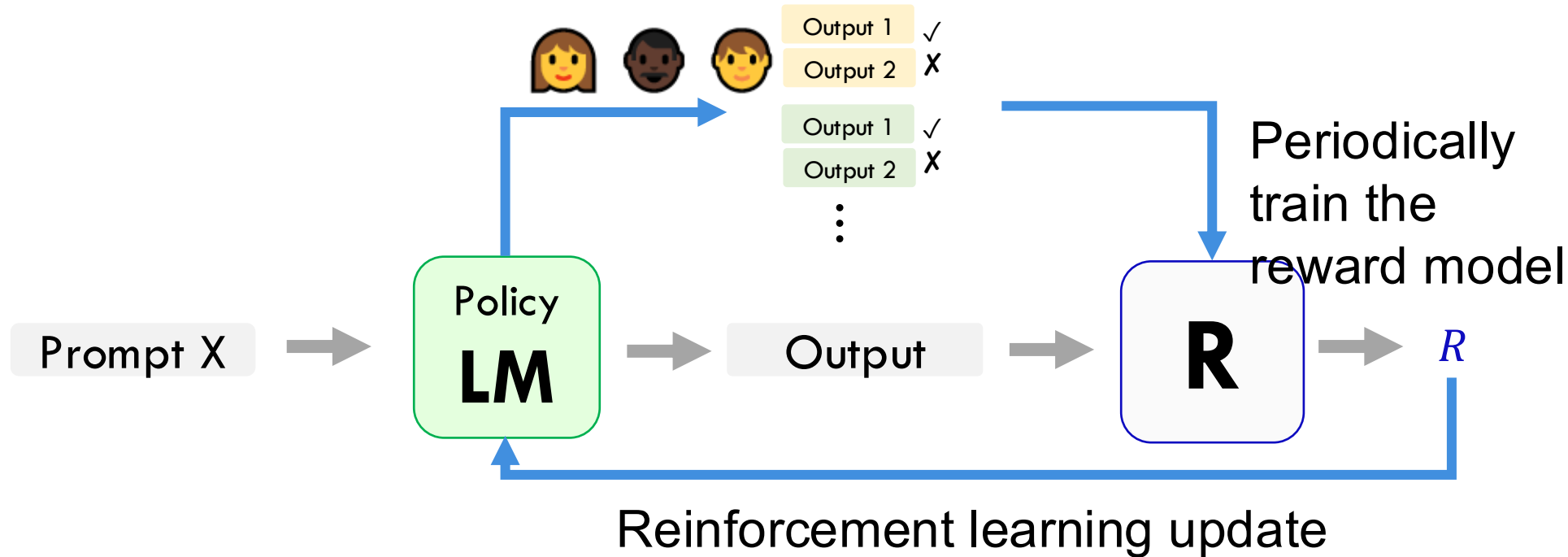
- Using this data, we can train a reward model
 - The reward model returns a scalar reward which should numerically represent the human preference.



Putting it Together (4)



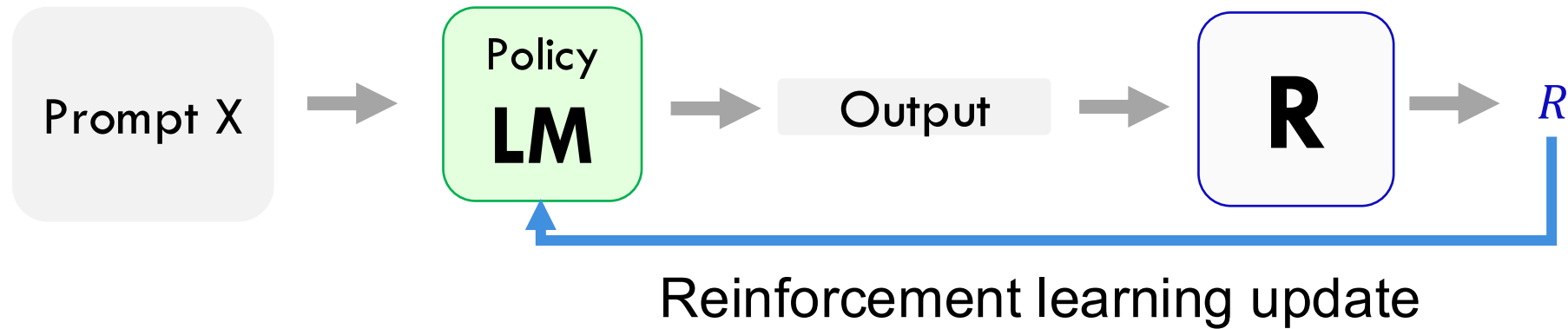
- Periodically train the reward model with more samples and human feedback



Putting it together (3)



- We want to learn a policy (a Language Model) that optimizes against the reward model



$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^n R(s_i; \mathbf{p}) \nabla_{\theta} \log p_{\theta}(s_i; \mathbf{p})$$

The overall recipe (for models like GPT 3.5) 🍳



Summary: RLHF / RLVR

- RL can help mitigate some of the problems with supervised instruction tuning
- RLHF uses two models
 - Reward model is trained via ranking feedback of humans.
 - Policy model learns to generate responses that maximize the reward model.
- Limitations:
 - RL can be tricky to get right
 - Training a good reward may require a lot of annotations